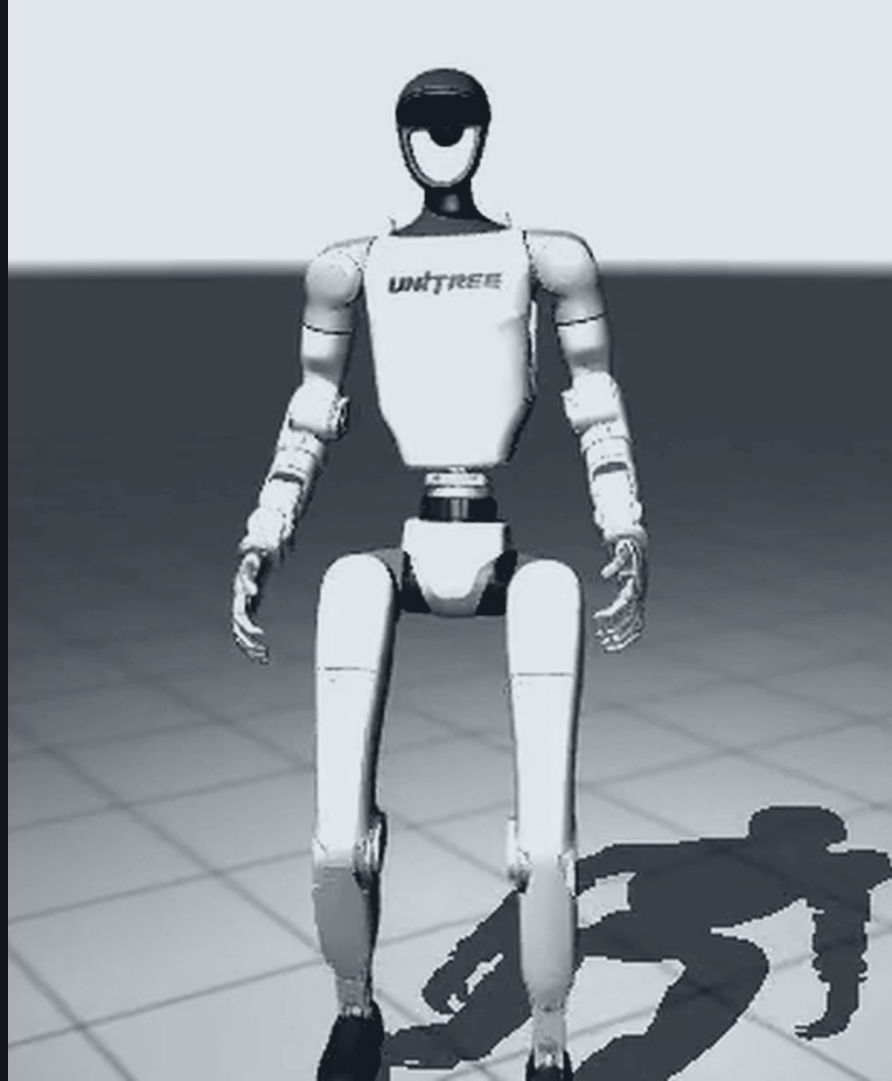
LINUX DAY 2026 · ROMA2LUG

Cadi, rialzati, cammina

Un robot umanoide impara a camminare con l'AI open source

Stefano Capezzone
Tor Vergata · 24 ottobre 2026



00 · IL PERCORSO DI STASERA

Dal robot che crolla al robot che cammina

01
Com'è fatto un umanoide

02
Il protagonista: Unitree G1

03
Il cervello: una rete neurale

04
Come impara: reinforcement learning

05
Perché il simulatore

06
Demo del simulatore

07
Come abbiamo organizzato il training

08
Cadi, rialzati, cammina

09
Cosa abbiamo imparato

00 · DAL VIVO

Lanciamo l'addestramento. Adesso.



Un G1 nuovo, con la rete a pesi casuali, comincia a imparare su una GPU in cloud. Lo lasciamo lavorare mentre parliamo.

8.192
robot simulati in parallelo

1 GPU
NVIDIA A100 su Google Colab

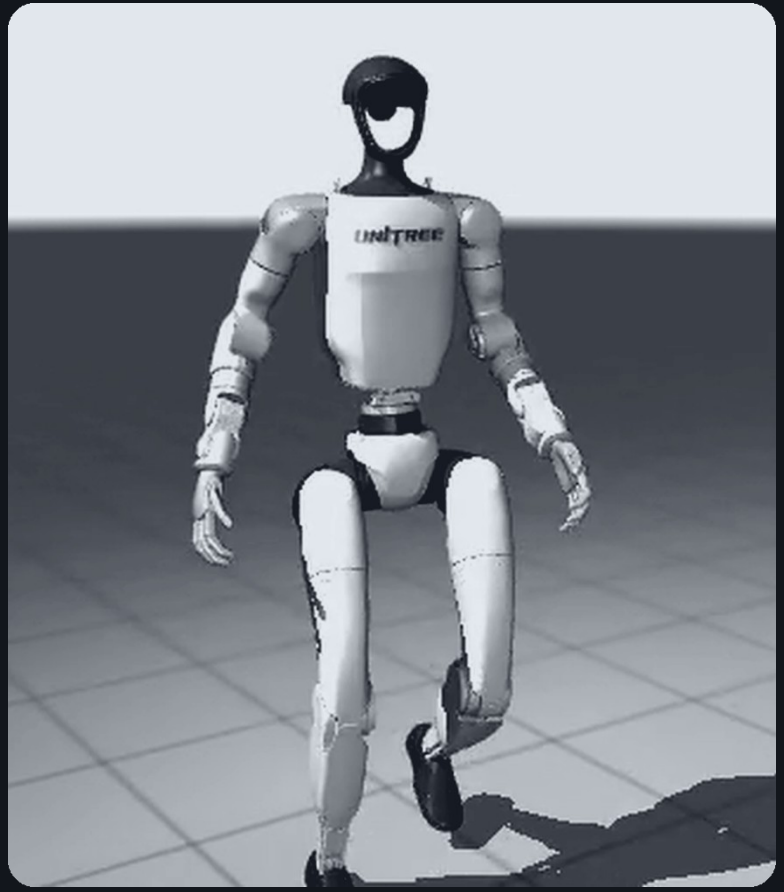
~35 min
di addestramento, fino alla fine del talk

Ora: premiamo invio → **Sezione 07:** guardiamo i progressi → **Fine:** cosa ha imparato

01

Com'è fatto un robot umanoide

Motori, sensori, controller



Tre ingredienti: muscoli, sensi, cervello



Motori

Uno per articolazione. Ricevono una posizione obiettivo e la raggiungono con la forza (coppia) necessaria.



Sensori

IMU: inclinazione e rotazione, come l'orecchio interno.
Encoder: l'angolo di ogni giunto.
Telecamere e LiDAR: il mondo intorno.

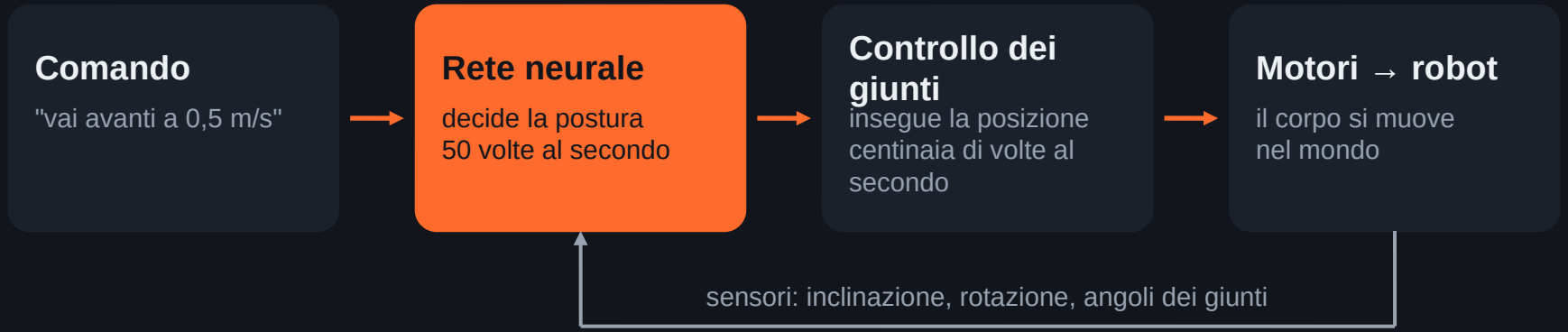


Controller

Legge i sensori, decide, comanda i motori. Decine di volte al secondo, senza pausa.

Camminare è una caduta controllata, corretta di continuo.

Un controllo a più livelli

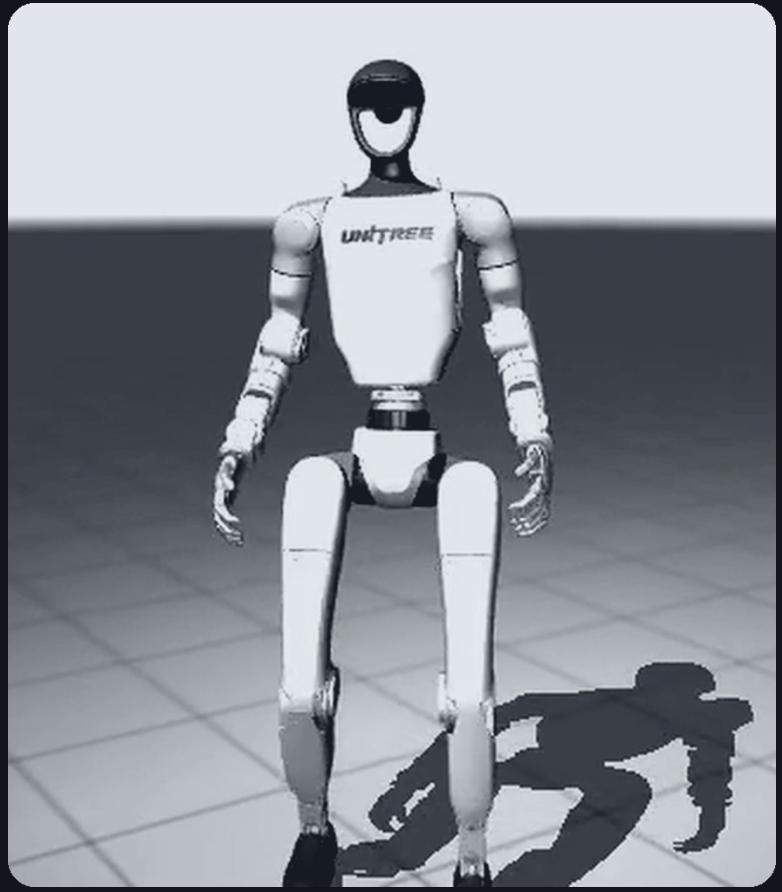


La rete neurale sta qui: è il livello che decide.

02

Il protagonista: Unitree G1

Un umanoide vero, con un gemello digitale fedele



Il robot

1,32 m
altezza

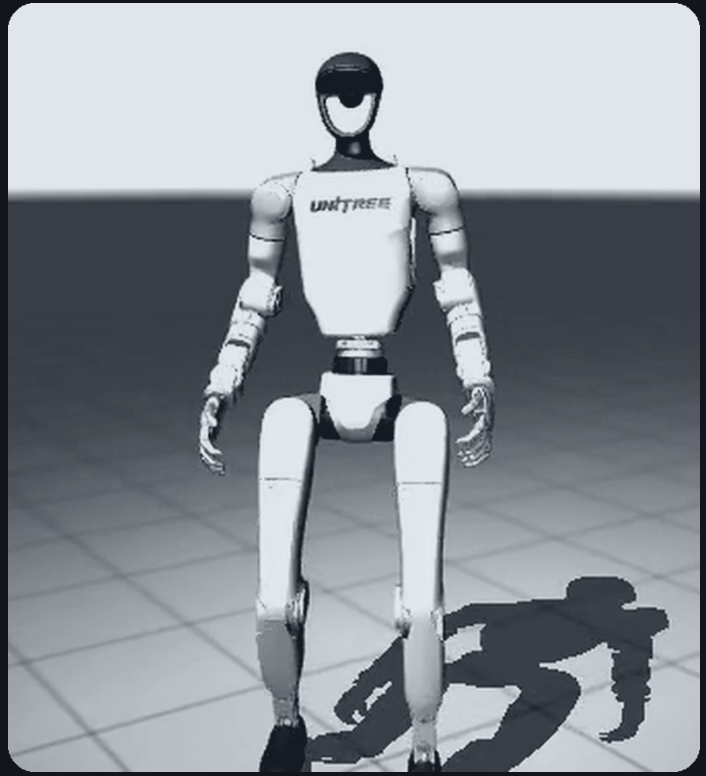
~35 kg
peso con batteria

23–43
gradi di libertà secondo la versione (il nostro modello: 29)

13.500 $
prezzo di partenza (listino)

Sensori: telecamera di profondità, LiDAR 3D, IMU, doppio encoder per giunto.

Perché lui: ha un modello ufficiale e fedele nel simulatore.

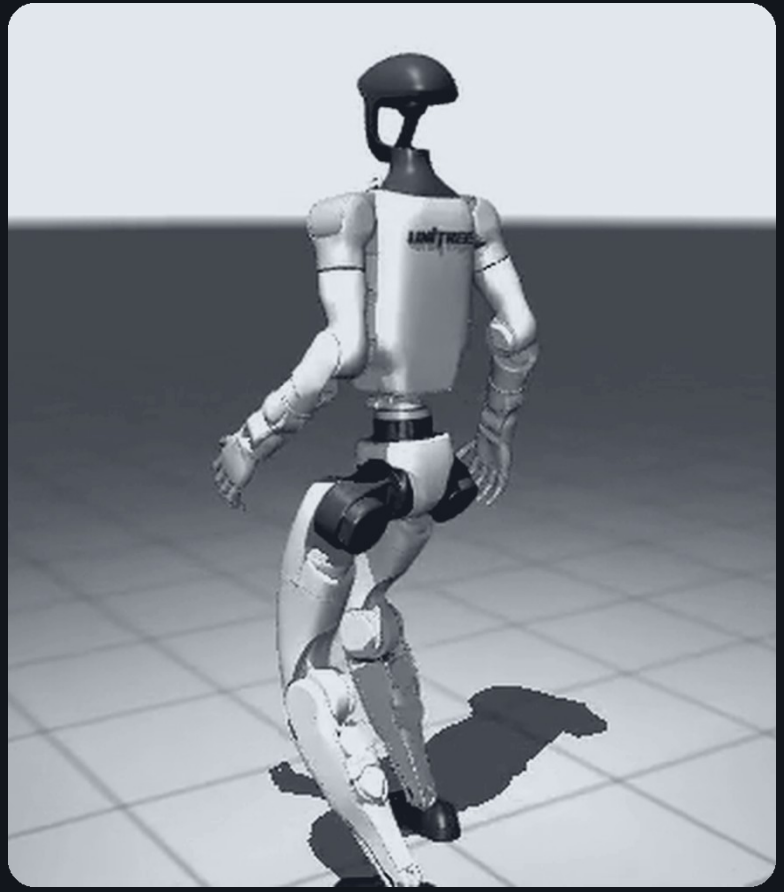


Fonte: unitree.com/g1

03

Il cervello: una rete neurale

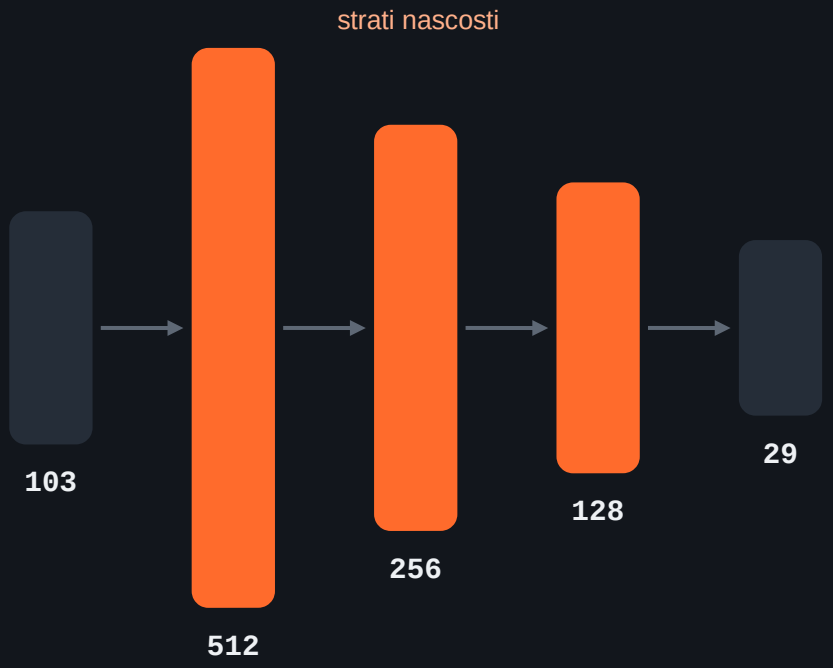
Cosa vede, cosa decide, quanto è grande



103 numeri entrano, 29 escono

ENTRANO · 103

- **3** velocità del corpo
- **3** rotazione (giroscopio)
- **3** direzione della gravità
- **3** comando: avanti, lato, gira
- **29** angoli dei giunti
- **29** velocità dei giunti
- **29** ultima azione
- **4** fase del passo



ESCONO · 29

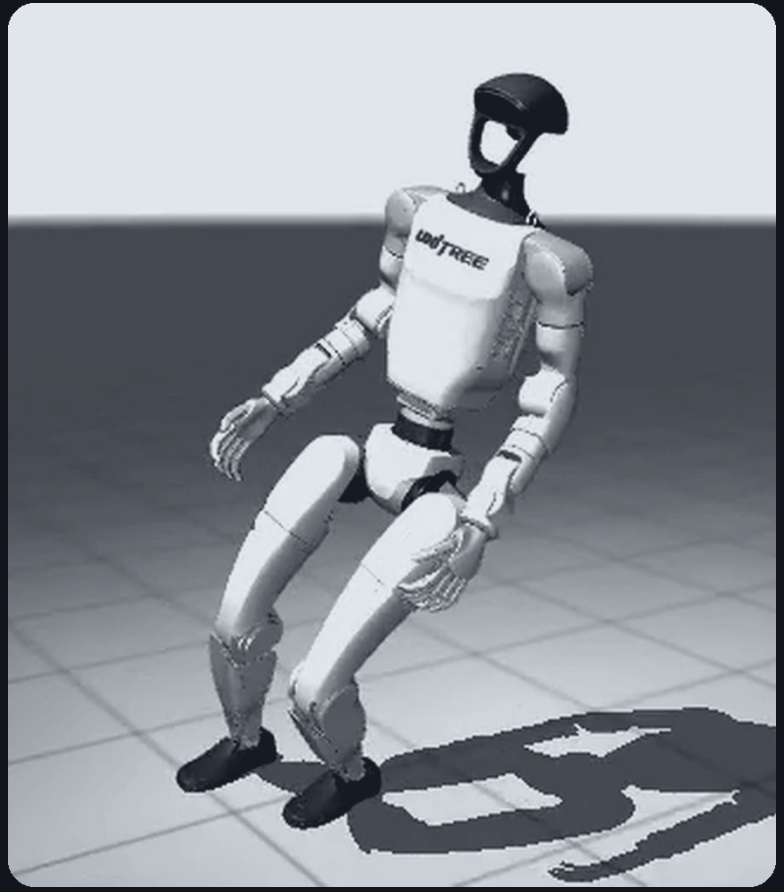
Una posizione obiettivo per ogni motore, 50 volte al secondo.

~225 mila
parametri: meno di 1 MB, gira su un portatile

04

Come impara: reinforcement learning

Premi, penalità e milioni di tentativi



Nessuno gli mostra come si fa



Ripetuto centinaia di milioni di volte. L'algoritmo (PPO) aggiusta i pesi della rete per rendere più probabili le azioni che hanno portato più premio.

Come addestrare un cane

Non gli spieghi come sedersi. Aspetti che lo faccia, e lo premi. Il cane non capisce la regola: capisce cosa porta il premio.

Il vero lavoro: decidere cosa premiare

Premi

- seguire la velocità richiesta
- seguire la rotazione richiesta
- fare passi veri, alzando i piedi

Penalità

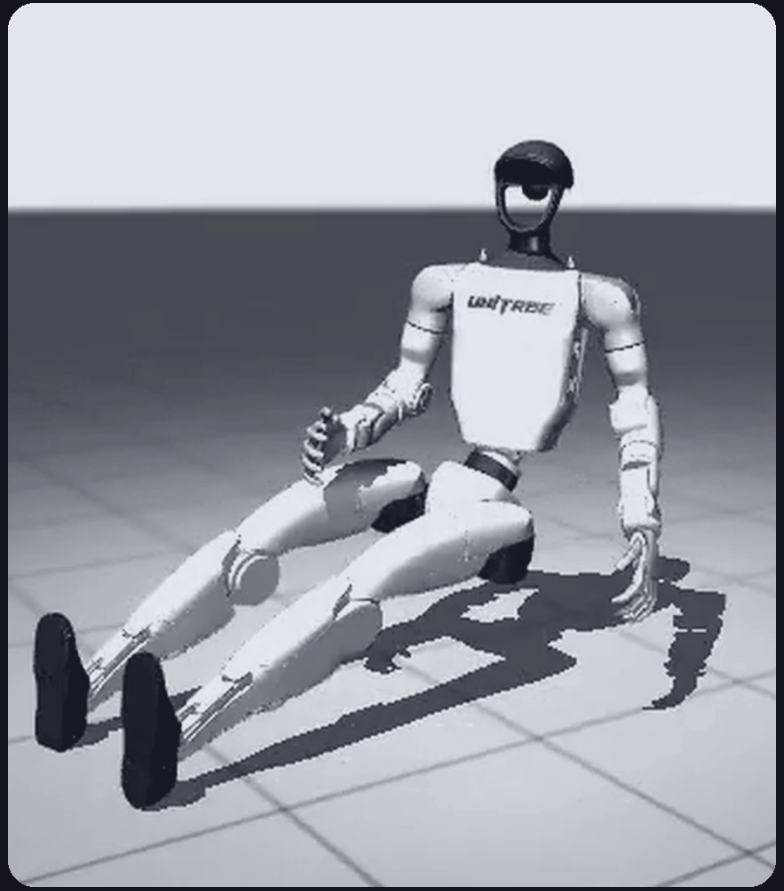
- cadere
- busto inclinato
- movimenti bruschi e scatti
- troppa forza, troppa energia
- piedi che scivolano

Il robot ottimizza quello che chiedi, non quello che intendi. Tenetelo a mente.

05

Perché il simulatore

Cadere, in simulazione, è gratis



05 · I NUMERI

Un laboratorio dove il tempo corre più veloce

8.192

robot che imparano in parallelo

203 M

passi di addestramento

~47 giorni

di esperienza, fatti in 2 ore

0

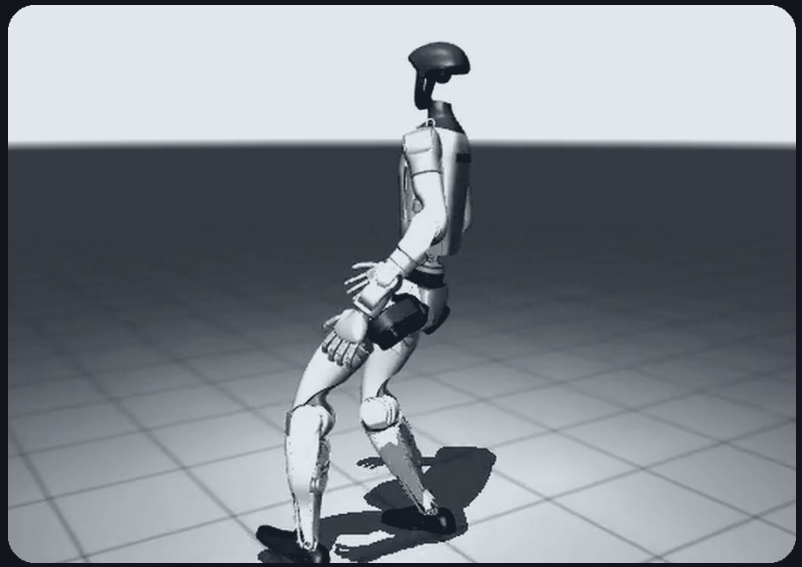
robot rotti

Un robot vero che cade migliaia di volte si rompe, e impara in tempo reale. In simulazione migliaia di copie cadono in parallelo, senza danni, molto più veloci del tempo reale.

Il simulatore sbaglia: rendiamolo imprevedibile

Il mondo vero non è mai identico al modello. Allora, a ogni tentativo, si cambiano le condizioni a caso (domain randomization): la rete impara a cavarsela in tutte.

- attrito del pavimento
- masse e baricentro
- attrito interno dei giunti
- spinte improvvise
- rumore sui sensori



Pavimento ghiacciato (attrito 0,2 invece di 0,8): il robot addestrato non cade.

Tutto open source, tutto su Linux

I nostri script	Python: addestramento, video, montaggio
MuJoCo Playground	ambienti pronti, G1 compreso
Brax	algoritmo PPO
MuJoCo + MuJoCo Warp	la fisica, anche su GPU
JAX	calcolo veloce su GPU
Linux	sotto a tutto



Sviluppati in gran parte da Google DeepMind e rilasciati con licenza libera.

L'unica cosa non libera: la GPU a noleggio (Google Colab).

06

Demo: il simulatore dal vivo

Il G1 senza cervello

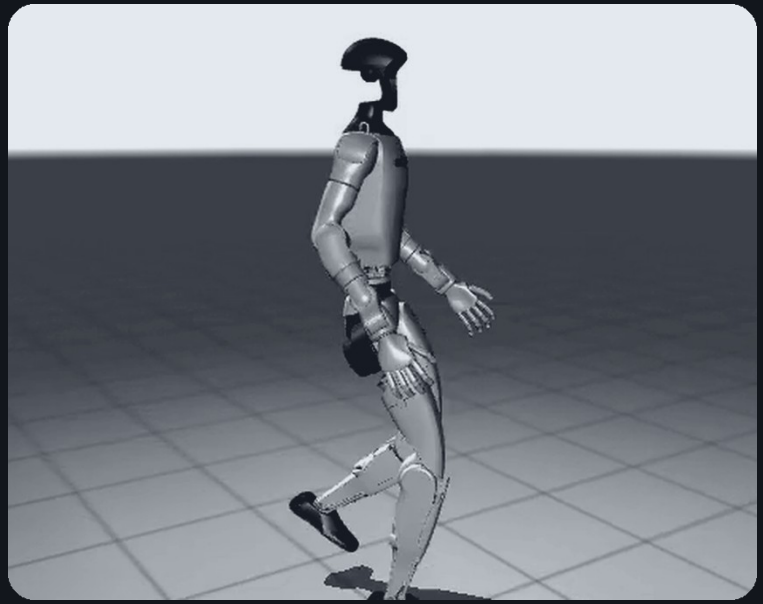
motori spenti: crolla

Accendiamo la rete

quella addestrata: sta in piedi e cammina

Proviamo a spingerlo

col mouse: reagisce e recupera



Visualizzatore MuJoCo sul portatile

07

Come abbiamo organizzato il training

E torniamo alla run lanciata all'inizio



Dal notebook al video

Notebook Colab

script Python e parametri PPO di riferimento DeepMind

GPU A100

8.192 robot, ~1,6 M passi al minuto

Fasi salvate

la rete a tappe, su Drive

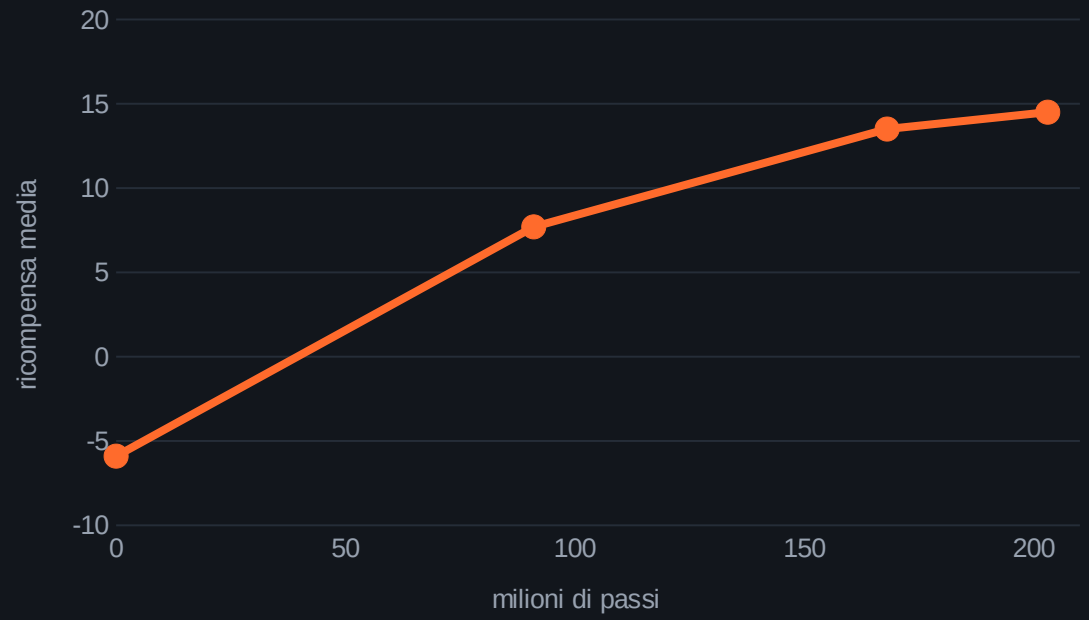
Video

rendering di ogni tappa, poi montaggio con ffmpeg

La nostra camminata: **203 M passi · 124 minuti · ~1,60 €**

Prima del G1, un riscaldamento con un umanoide stilizzato più semplice.

La ricompensa sale, le cadute spariscono



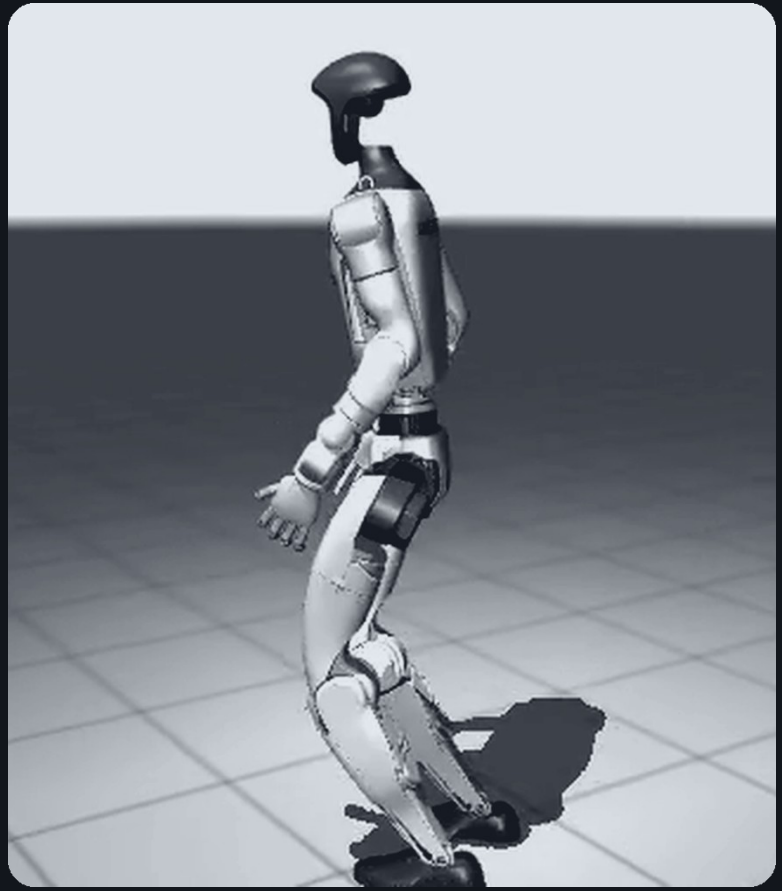
-5,9 → 14,5
ricompensa media, da 0 a 203 M passi

40 → ~750
passi prima di cadere, su 1.000 (con spinte)

08

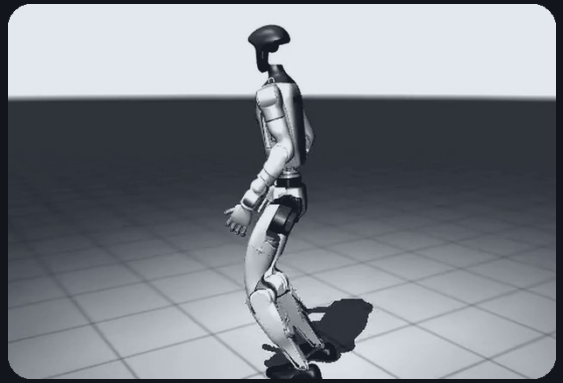
Il nostro percorso

Cadi, rialzati, cammina: tre atti, con i video



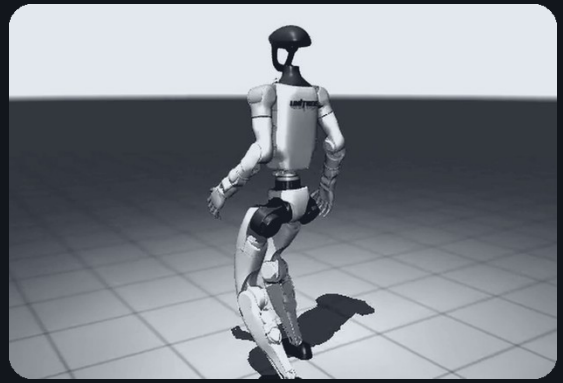
08 · ATTO 1

Cammina



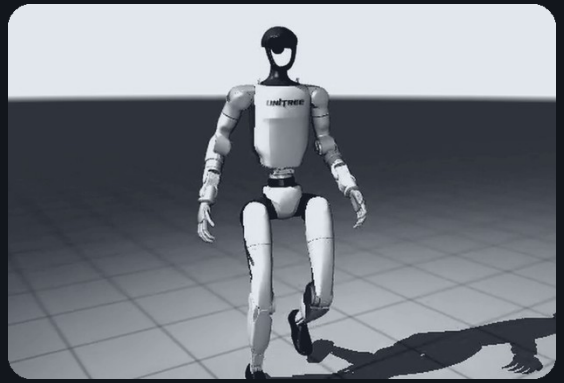
0 passi

rete a caso: crolla



30 M passi · ~20 min

ci prova, a volte cade



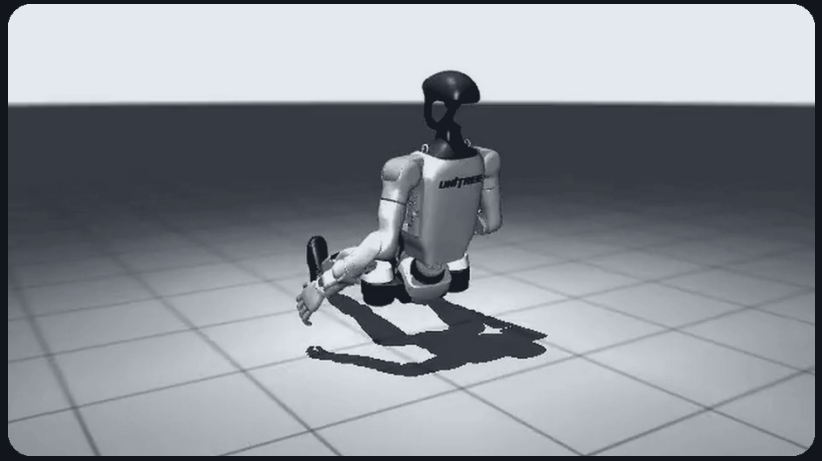
203 M passi · 2 ore

cammina, curva, va di lato

clip 1, 2, 4

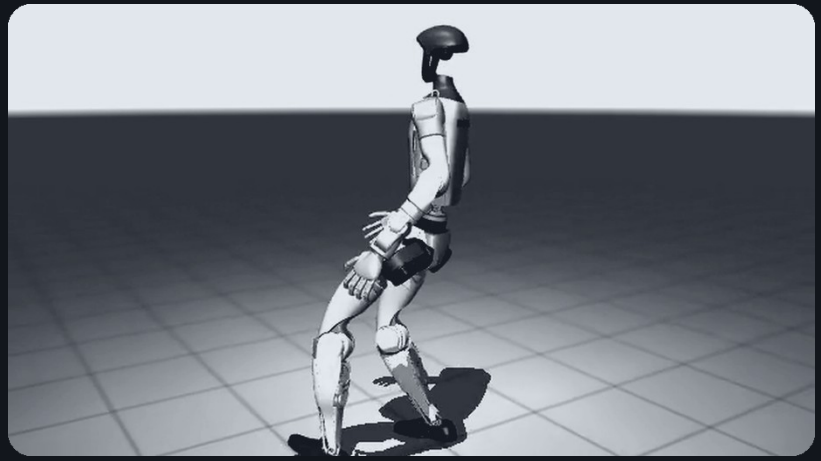
08 · ATTO 2

Resiste



30 M passi, gli chiediamo 1,8 m/s

capitombolo



Ghiaccio: attrito 0,2 invece di 0,8

il robot addestrato non cade

clip 3, 5

Rialzati: il robot furbo

Tentativo 1

"Ti premio se alzi il busto"

Si puntella sui gomiti. Busto alto, missione compiuta.

Tentativo 2

"Ti premio se stai in alto"

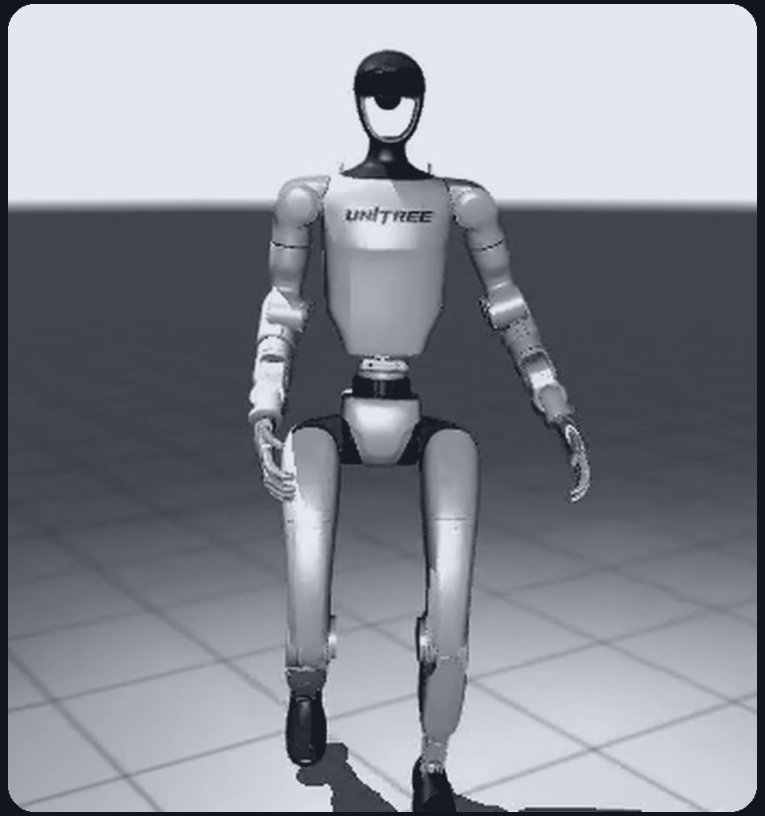
Si siede composto: bacino a 0,43 m. In piedi sarebbe 0,755.

Tentativo 3

Diagnosi e aiuto

Non poteva piegare abbastanza le ginocchia. Escursione completa e una "mano" che lo tira su, tolta a poco a poco: si alza in ~1 s.

450 M passi e ~5 ore per imparare a sedersi · altri 450 M e ~5 ore per alzarsi



LA LEZIONE

Il robot fa quello che gli chiedi, non quello che intendi.

Si chiama reward hacking. Scrivere la ricompensa giusta è metà del lavoro: l'altra metà è guardare i video e capire cosa ha imparato davvero.

09 · DAL VIVO

Dove è arrivata la nostra run?

Passi fatti

?

attesi: ~35-40 M

Ricompensa

?

confronto con la curva

Tempo e costo

?

minuti di A100, unità Colab

Passiamo al notebook.

Cosa portiamo a casa

1 **Si può fare con poco**
un umanoide che cammina: 2 ore di GPU, 1,60 €, tutto software libero.

2 **Il simulatore è il laboratorio**
cadere milioni di volte non costa niente.

3 **Il difficile è la ricompensa**
il robot trova sempre la scorciatoia.

I limiti

- non l'abbiamo provato su un G1 vero
- passare dal simulatore alla realtà resta la parte più difficile
- per rialzarsi è servito un aiuto

RIFATELO A CASA

github.com/google-deepmind/mujoco_playground

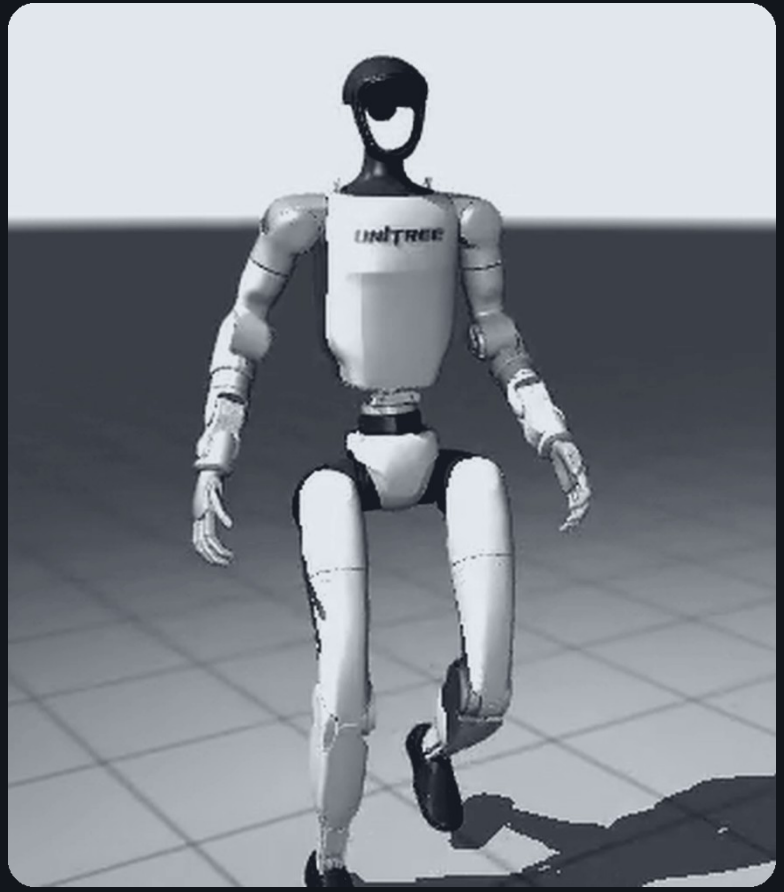
mujoco.org

github.com/capezzone/cadi-rialzati-cammina

stefano@capezzone.it

Grazie!

Domande?



A · PPO IN UN GIRO

Come impara: il ciclo di PPO

1 Raccogli

8.192 robot giocano 20 passi ciascuno con la rete di adesso: 163.840 esempi (stato, azione, ricompensa).

→

2 Giudica

Il critico stima quanto vale ogni stato. Con GAE si calcola il vantaggio: è andata meglio o peggio del previsto?

→

3 Aggiorna

4 passate sui dati in 32 minibatch: 128 passi di Adam. Il clip impedisce alla rete di cambiare troppo in un colpo.

→

4 Ripeti

I dati si buttano e si riparte con la rete nuova (on-policy). 203 M passi = ~1.240 giri, ~159.000 aggiornamenti.

Proximal = vicino: ogni aggiornamento resta vicino alla politica che ha raccolto i dati. È questo che rende PPO stabile, semplice e il cavallo di battaglia della robotica in simulazione.

A · LA FUNZIONE OBIETTIVO

Cosa ottimizza PPO, termine per termine

RAPPORTO

$$\rho_t = \pi_\theta(a_t|s_t) \,/\, \pi_{old}(a_t|s_t)$$

OBIETTIVO CON CLIP

$$L^{clip} = \mathbb{E}\left[\min\left(\rho_t \hat{A}_t \,,\; \mathrm{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t \right) \right]$$

ERRORE DEL CRITICO

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

VANTAGGIO (GAE)

$$\hat{A}_t = \sum_l (\gamma\lambda)^l \cdot \delta_{t+l}$$

PERDITA TOTALE

$$L = -L^{clip} + c_v (V - V_{target})^2 - c_e H[\pi]$$

$\varepsilon = 0{,}2$
quanto può cambiare la politica in un giro

$\gamma = 0{,}97$
sconto: orizzonte ≈ 1/(1−γ) ≈ 33 passi ≈ 0,7 s

$\lambda = 0{,}95$
GAE: compromesso tra rumore e distorsione

$c_v = 0{,}5$
peso dell'errore del critico

$c_e = 0{,}005$
bonus di entropia: tiene viva l'esplorazione

$\hat{A}$ normalizzati
media 0 e varianza 1 per minibatch

A · ATTORE E CRITICO

Due reti: una guida, l'altra giudica

ATTORE — va sul robot

Ingressi 103: velocità, giroscopio, gravità, comando, angoli e velocità dei 29 giunti, ultima azione, fase del passo (con rumore)

Rete MLP 512 → 256 → 128, attivazione swish

Uscite 58 = 29 medie + 29 deviazioni standard (distribuzione tanh-normale)

Azione posizione obiettivo = posa di riferimento + 0,5 · tanh(campione), 50 volte al secondo

Dimensione ~225.000 parametri: meno di 1 MB

CRITICO — solo in addestramento

Ingressi 216 "privilegiati": i 103 dell'attore + gli stessi valori senza rumore, accelerometro, altezza del bacino, forze dei motori, contatti e velocità dei piedi

Rete MLP 512 → 256 → 128 → 1, swish

Uscita V(s): quanta ricompensa ci si aspetta da qui in avanti

Dimensione ~275.000 parametri

Asimmetrico: il critico può "barare" perché sul robot vero non ci va. Ingressi normalizzati con media e varianza mobili (Welford).

A · I NUMERI DELLA RUN

Iperparametri: quelli di DeepMind, senza ritocchi

SIMULAZIONE

Ambiente	**G1JoystickFlatTerrain**
Simulatore	**MuJoCo Warp su GPU**
Robot in parallelo	**8.192**
Controllo / fisica	**20 ms (50 Hz) / 2 ms**
Episodio	**1.000 passi = 20 s, stop se cade**
Passi totali	**203 M (≈ 47 giorni simulati)**
Valutazioni	**41, su 128 robot ciascuna**
Hardware e tempo	**1 × NVIDIA A100, 124 min**
Costo	**~14 unità Colab ≈ 1,60 €**

PPO

Algoritmo	**PPO di Brax 0.14.2**
Rollout	**20 passi per robot**
Minibatch	**32 × 256 sequenze**
Epoche per giro	**4**
Ottimizzatore	**Adam, lr 0,0003 costante**
Clip ε	**0,2**
Sconto γ / GAE λ	**0,97 / 0,95**
Entropia / critico	**0,005 / 0,5**
Clip del gradiente	**norma 1,0**

A · RICOMPENSA E REALISMO

La ricompensa del G1 e il mondo imprevedibile

RICOMPENSA: PESI

seguire la velocità richiesta	+1,0
seguire la rotazione richiesta	+0,75
piedi in aria (passi veri)	+2,0
piedi a tempo con la fase	+1,0
caduta (fine episodio)	−100
busto inclinato	−2,0
muoversi quando deve stare fermo	−1,0
giunti a fine corsa	−1,0
piedi che scivolano	−0,25
anche lontane dalla posa	−0,25
oscillazioni del busto	−0,15
ginocchia, posa, autocollisioni	−0,1
forze di contatto eccessive	−0,01

DOMAIN RANDOMIZATION

Attrito piedi–pavimento
0,4 – 1,0

Attrito interno dei giunti
× 0,9 – 1,1

Inerzia dei motori (armatura)
× 1,0 – 1,05

Masse dei segmenti
× 0,9 – 1,1; busto ± 1 kg

Posa di zero dei giunti
± 0,05 rad

Spinte
ogni 5–10 s, 0,1–2 m/s

Rumore sui sensori
giunti ± 0,03 rad, giroscopio ± 0,2 rad/s, …

Comandi casuali (ogni 10 s)
avanti ± 1 m/s, lato ± 0,5 m/s, rotazione ± 1 rad/s

A · RIFERIMENTI

Per approfondire

ARTICOLI

Schulman et al., 2017 Proximal Policy Optimization Algorithms — arXiv:1707.06347

Schulman et al., ICLR 2016 High-Dimensional Continuous Control Using Generalized Advantage Estimation — arXiv:1506.02438

Huang et al., ICLR Blog Track 2022 The 37 Implementation Details of Proximal Policy Optimization

Pinto et al., 2017 Asymmetric Actor Critic for Image-Based Robot Learning — arXiv:1710.06542

Tobin et al., 2017 Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World — arXiv:1703.06907

Rudin et al., CoRL 2021 Learning to Walk in Minutes Using Massively Parallel Deep RL — arXiv:2109.11978

Freeman et al., 2021 Brax: A Differentiable Physics Engine for Large Scale Rigid Body Simulation — arXiv:2106.13281

Zakka et al., RSS 2025 MuJoCo Playground — arXiv:2502.08844

CODICE E CORSI

github.com/google-deepmind/mujoco_playground

github.com/google/brax

mujoco.org

spinningup.openai.com — PPO

huggingface.co/learn/deep-rl-course

Le nostre versioni

playground 0.2.0 · brax 0.14.2 · jax 0.9.2 · flax 0.12.6 · MuJoCo 3.13 + Warp 1.17